

1.

```
/**
 * Inputs a "menu" of words (items) and their prices from the given file and
 * stores them in the given {@code Map}.
 *
 * @param fileName
 *         the name of the input file
 * @param priceMap
 *         the word -> price map
 * @replaces priceMap
 * @requires <pre>
 * [file named fileName exists but is not open, and has the
 *  format of one "word" (unique in the file) and one price (in cents)
 *  per line, with word and price separated by ','; the "word" may
 *  contain whitespace but no ',']
 * </pre>
 * @ensures [priceMap contains word -> price mapping from file fileName]
 */
private static void getPriceMap(String fileName,
                                Map<String, Integer> priceMap) {
    priceMap.clear();
    SimpleReader fileReader = new SimpleReader1L(filename);
    While( !fileReader.atEOS()){
        String line = fileReader.nextLine();
        Int index = line.indexOf(',');
        If( index != -1){
            String name = line.substring(0, index).trim();
            Int price = Integer.parseInt(line.substring(index + 1).trim());
            priceMap.add(name, price);
        }
    }
    fileReader.close;
}
```

2.

```
/**
 * Input one pizza order and compute and return the total price.
 *
 * @param input
 *         the input stream
 * @param sizePriceMap
 *         the size -> price map
 * @param toppingPriceMap
 *         the topping -> price map
```

```

* @return the total price (in cents)
* @updates input
* @requires <pre>
* input.is_open and
* [input.content begins with a pizza order consisting of a size
* (something defined in sizePriceMap) on the first line, followed
* by zero or more toppings (something defined in toppingPriceMap)
* each on a separate line, followed by an empty line]
* </pre>
* @ensures <pre>
* input.is_open and
* #input.content = [one pizza order (as described
* in the requires clause)] * input.content and
* getOneOrder = [total price (in cents) of that pizza order]
* </pre>
*/
private static int getOneOrder(SimpleReader input,
    Map<String, Integer> sizePriceMap,
    Map<String, Integer> toppingPriceMap) {
    Int total = 0;
    Boolean con = true;

    If(input.atEos()){
        String size = input.nextLine().trim();
        If(sizePriceMap.haskey(size)){
            Total = total + sizePriceMap.value(size);
        }
    }

    While(!input.atEos() && con){
        String topping = input.nextLine().trim();
        Con = !tpping.isEmpty();

        If(con && toppingPriceMap.haskey(topping){
            Total = total + toppingPriceMap.value(topping);
        }
    }

    Return total;
}

```